SYSTEM AN DEVICE PROGRAMMING PROJECT

Device Programming Part

Project Q1

Graph Neural Network Inference on GPUs

Elisa Turco s358652
Federico Pintabona s360688

Table of contents

1. Introduction and Project's Goal 2
2. Design Choices 2
   2.1 Implementation Details 2
   2.2 Graph Format and Memory Layout 3
   2.3 Dataset extraction and Preprocessing 4
3. Implementation Choices 4
   3.1 General Details 4
   3.2 Sequential Implementation 5
   3.3 Parallel CPU Implementation 5
      3.3.1 Vertex-centric 7
      3.3.2 Vertex-centric with Chunks 8
      3.3.3 Edge-centric with Chunks 9
   3.4 Parallel GPU Implementation 11
      3.4.1 Thread per Node with Global Memory 12
      3.4.2 Thread per Feature with Shared Memory 13
      3.4.3 Thread per Feature with Shared Memory and Feature Compression 15
      3.4.4 Thread per Edge with Global Memory 16
4. Experimental Evaluation 18
   4.1 Experimental Setup 18
      4.1.1 Hardware Specification 18
      4.1.2 Profiling Tools & Metrics 19
   4.2 Global Performance Analysis 19
      4.2.1 CPU vs GPU: Global Speedup 20
      4.2.2 CPU Parallelization Strategies 21
   4.3 GPU Architecture Evaluation 21
      4.3.1 Vertex-centric vs. Edge-centric 21
      4.3.2 Performance and Memory Footprint Evaluation 22
   4.4 Scalability Evaluation 23
      4.4.1 Number of Features 23
      4.4.2 Number of Nodes 24
5. Conclusions 24

1. Introduction and Project's Goal

Graph Neural Networks (GNNs) are a type of artificial intelligence designed to analyse data structured as graphs, which consist in nodes (objects) and edges (the connections between them).

They are used for studying complex connections and relationships, finding large usage in fields such as Social Networks, Recommender Systems, Cybersecurity and Fraud Detection**.**

The goal of this project is to develop and test and inference engine for GNNs, evaluating how different implementation and parallelization strategies affect performance and scalability on different types of graphs.

2. Design Choices

2.1 Implementation Details

For the development of the inference engine, the forward path (the mathematical process where the network processes the input graph data to generate the final representations) has been implemented based on the following design choices:

- **Weights initialization and Reproducibility**: to simulate pre-trained model weights, all weight matrices are randomly generated by random_weights.cpp. To guarantee reproducibility and accurate performance evaluation, the random number generator has a fixed seed, to obtain identical weight matrices across all executions.
- **Aggregation**: this inference engine uses a sum operator to compute aggregation. For each node, the engine sums the feature of all its neighbour. A self-loop is used to combine the current node state and the aggregated message, simply summing its current features to the neighbour's message.
- **Update**: the chosen approach is GCN-style, to maintain a constant feature dimension across layers and avoid increasing computational cost at each layer (unlike GraphSAGE, which concatenates the node's current state with the neighbour message, significantly increasing the vector's size).

  $$h_v^{(l+1)} = \sigma(W^{(l)} m_v^{(l)})$$

  The aggregated sum is multiplied by the layer's weight matrix. Finally, before saving the results into the next_features vector, a ReLU activation function is applied. ReLU is an extremely light non-linear function that breaks the linearity between layers. Without such a non-linear function, stacking layers would mathematically collapse into a single linear transformation, preventing the network from learning complex patterns. If a computed feature value is positive, the information is propagated; otherwise, its value is set to zero, meaning that specific feature is considered non-useful and is discarded.

  $$\max(0.0f, x)$$

- **Final layer and Logits**: the final layer is the classification layer. It performs the multiplication between the output features of the last hidden layer and specific output matrix called

weights_out (with dimensions num_features x num_classes). The results are raw scores called logits, one for each possible class.

- **Class assignment**: finally, to assign a predicted class to each node, the argmax operator picks the class index with the highest score.
- **Accuracy calculation and Dataset Splits**: when training a neural network, one cannot evaluate it using the same nodes it learned on. For this reason, nodes are divided into 3 groups, called "splits": training split (to train the network), validation split (to tune the network), and test split. The test split contains nodes the network has never seen during training, which are used to evaluate the true accuracy of the model. Since this project requires implementing an inference engine (considering an already trained network), only the test split has been used to calculate the final accuracy:

$$Accuracy = \frac{Correct\ predictions\ on\ Test\ Split}{Total\ nodes\ in\ test\ Split}\ x\ 100$$

2.2 Graph Format and Memory Layout

The storage format used for the graphs of this project is Compressed Sparse Row (CSR), which uses one-dimensional arrays to represent sparse matrix saving memory and speeding-up operations.

It was chosen over a Dense Matrix representation because large and sparse graphs (like real ones), contain many zeros, occupying RAM with useless information.

CSR uses two main arrays:

- **Indices array** acts as an adjacency matrix. It contains 32-bit integers, that are the neighbour's ids for each source node, in sequential order.
- **Indptr array** saves where a node's neighbours start and end (end index excluded).

All data is allocated as 1D dense vectors rather than dense matrices with multiple pointers. This choice was initially driven by the need to use the CSR arrays (indptr and indices) and to simplify CUDA memory management.

Furthermore, this layout provides performance benefits on both architectures. On the CPU, it guarantees *spatial locality*. Since the processor loads contiguous memory blocks, using 1D arrays minimizes the number of cache misses. On the GPU, this structure allows threads to perform *coalesced memory accesses*, maximizing the VRAM bandwidth.

Apart from CSR arrays, the other ones used are:

- **Features array** represent the [num_nodes x num_features] dense matrix.
- **Labels array** is a num_nodes long array that indicates for each node the right class.

Last, meta.txt contains information about the number of nodes, edges, features and classes. It is read to populate an unordered map, used to allocate the required memory vectors.

2.3 Dataset extraction and Preprocessing

The project requires to use both real and synthetic graphs. For this reason, three specific scripts have been written:

- **csr_extraction.py** uses the *OGB* (Open Graph Benchmark) Python library to download real datasets (such as ogbn-arxiv) and extract their features and edge list. Then, *scipy.sparse* (a mathematical library) automatically calculates the indptr and indices arrays from the raw edge list. The output of this script consists of the raw .bin files (indptr, indices, features, labels and splits) and the meta.txt metadata file.
- **generate_synthetic.py** uses the *networkx* Python library that contains built-in algorithms to generate Scale-Free, Random and Small-World graphs. It relies on *scipy.sparse* in the exact same way as the previous script, producing the same output files.
- **bin_conversion.cpp** reads the .bin files and writes the data into the vectors used by main program.

3. Implementation Choices

3.1 General Details

The main.cpp module directs the entire program. When executed, it provides an interactive menu to select the desired Dataset and Algorithm to test. The initialization phase exploits the binary reading functions to populate the previously introduced vectors. Weights are deterministically generated (as explained before). The setup time is intentionally left out of the performance evaluation to measure the effective inference time.

For every implementation described below, the flow is the same. An outer loop iterates over the number of layers and takes the branches based on the type of the current layer:

- *Hidden layer*: if the current layer is not the last one, the inference engine performs the default message passing (Aggregation, Update and ReLU).
- *Double Buffering*: to avoid race conditions and incorrect results during parallel execution, threads read values from the feature vector and write results on a different vector, next_features. Only once all nodes have been processed by the current layer, the feature vector is updated with the new states of nodes.
- Final layer: the engine computes the Logits to generate the final decisions.

Execution timers have been implemented with *std::chrono* and only wrap around the actual inference engine algorithm, excluding other phases such, I/O operations, allocations and Accuracy calculation. Once the inference engine has stopped, the main evaluates its performance by plotting the Accuracy percentage (calculated on the test split) and the time it required to be computed. The test split and the labels are only loaded at this final stage.

3.2 Sequential Implementation

This represents the baseline implementation of the project. It contains the raw mathematics to perform the Aggregation, Update, ReLU, and to compute the Final layer. It is fundamental both to test the mathematical correctness of the inference engine and to act as a reference point to evaluate the speed-up introduced by the parallel versions.

The program is executed entirely on a single CPU core. Within the main layer loop, an outer loop goes through every node. For each node, an inner loop uses the *indptr* array to determine the start and the end of its neighbors. Then, it uses the *indices* array to access and sum the features of each neighbor. The aggregated message is represented by the local buffer *vsum*, then multiplied by the weight matrix.

```
        // loop through nodes
        for (int j=0; j<num_nodes; j++)
        {
            // reset the buffer
            fill(vsum.begin(), vsum.end(), 0.0f);

            // self-loop
            for (int k=0; k<num_features; k++)
            {
                vsum[k] += features[j*num_features + k];
            }

            // add neighbors' features
            for (int c=indptr[j]; c<indptr[j+1]; c++)
            {
                for (int k=0; k<num_features; k++)
                {
                    vsum[k] += features[indices[c]*num_features + k];
                }
            }

            // update and ReLU
            for (int r = 0; r<num_features; r++)
            {
                float sum = 0.0f;
                for (int c=0; c<num_features; c++)
                {
                    sum += weights[r*num_features+c] * vsum[c];
                }
                next_features[j*num_features + r] = max(0.0f, sum);
            }
        }
        features = next_features;
    }
```

Figure 1 Program Flow

3.3 Parallel CPU Implementation

To implement the parallel CPU versions of the inference engine, the standard *std::thread* C++ library was used.

A first thought would be to create a thread for each node or a fixed number of threads significantly larger than the number of available CPU cores. However, this ‘primitive’ parallel approach would cause a severe performance degradation due to *Context Switching*. This is a phenomenon where the OS constantly (at intervals of few milliseconds) flushes registers and cached data, pausing and switching between the threads assigned to a core.

To avoid this useless overhead, the solution adopted was using a **ThreadPool class**. The number of worker threads isn't an a-priori fixed integer; instead, its value depends on the hardware running the program, thanks to the *std::thread::hardware_concurrency()* method, which returns the number of logical CPU cores. In this way, there are as many threads as the working cores, and each core works 100% of the time on one single thread, without pausing it. The threads simply catch tasks to be done from a shared queue; to avoid race conditions, a mutex and a condition variable allow worker threads to safely dequeue tasks.

```
19  class ThreadPool{
20
21      private:
22          vector<thread> workers;
23          queue<function<void()>> tasks;
24          bool stop=false;
25          mutex mtx;
26          condition_variable cv;
27
28      public:
29          ThreadPool()
30          {
31              for (int i=0; i<thread::hardware_concurrency(); i++)
32              {
33                  workers.emplace_back([this] (){
34                      while (true)
35                      {
36                          function<void()> task;
37                          {
38                              unique_lock<mutex> Lock(this->mtx);
39                              this->cv.wait(Lock, [this](){ return this->stop || !this->tasks.empty();});
40                              if (this->stop && this->tasks.empty())
41                              {
42                                  return;
43                              }
44
45                              task = std::move(this->tasks.front());
46                              this->tasks.pop();
47
48                          }
49                          task();
50                      }
51                  });
52              }
53          }
54
```

Figure 2 ThreadPool class

Alongside the worker threads, there is one *Main thread* that is in charge of enqueuing the tasks. It puts tasks into the queue using the mutex and then waits while the workers process them. Once a worker thread has finished a task, it looks back at the queue; if there are undone tasks, it picks one up and executes it on its own core.

```
55          void enqueue(function<void()> task)
56          {
57              {
58                  unique_lock<mutex> Lock(this->mtx);
59                  this->tasks.push(task);
60              }
61              this->cv.notify_one();
62          }
63
```

Figure 3 Function used by the Main Thread to enqueue tasks

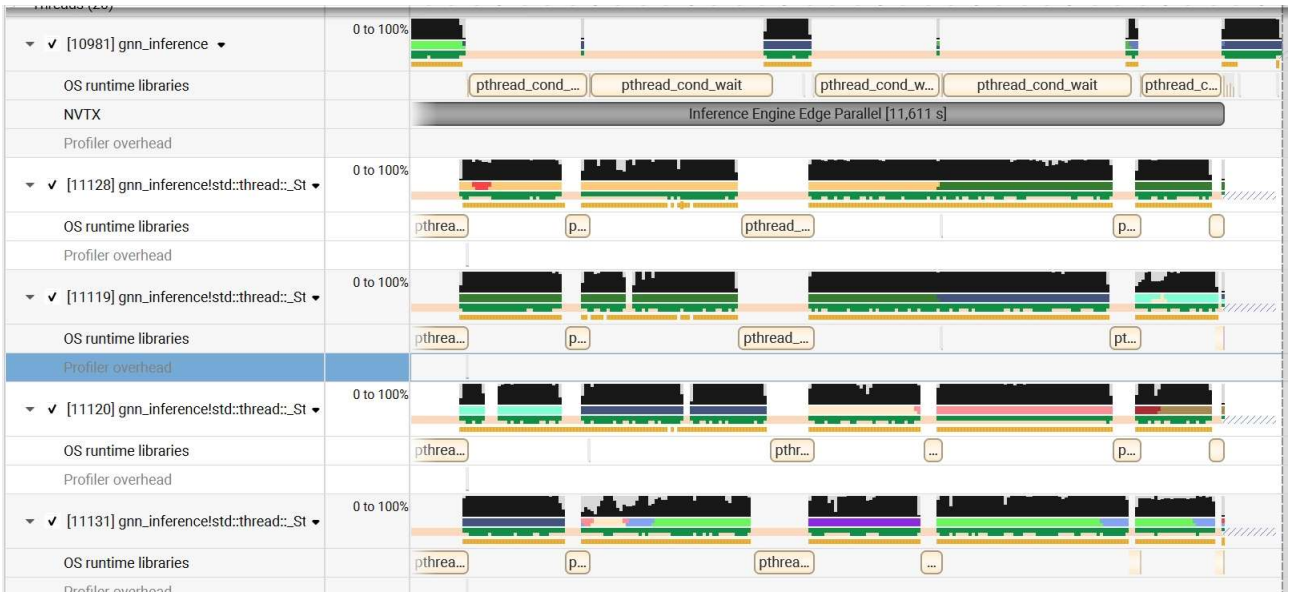


Figure 4 Nsight Systems CPU timeline illustrating the Thread Pool behavior: the Main Thread enters a sleep state while the worker threads process the enqueued tasks

Given this architectural choice, three strategies were explored and implemented:

- **Vertex-centric**
- **Vertex-centric with Chunks**
- **Edge-centric with Chunks**

The **Layer Parallelism** solution was intentionally discarded. Dealing with a single (and large-scale) graph, parallelizing across layers was highly impractical. Two strategies were explored, and discarded:

- Calculate each node's update as soon as all of its neighbors have been elaborated by the previous layer, even if the previous layer has not finished yet. This strategy was discarded due to the topology of the graph, since it would result in a waterfall of neighbors and a chain of promises and futures.
- Partitioning the graph into smaller ones, assigning a specific CPU core to each layer. This method was unfeasible since it requires a complex manual graph fragmentation into batches trying to minimize edge cuts (which means splitting two neighbours in two different batches). Even if one were to replicate the neighbour node into two different batches (simulating the right connection with *ghost nodes*), it would be an extremely complex and expensive task.

In conclusion, a layer-parallelization strategy would have efficiently done the work elaborating multiple independent graphs, which is not the case for this project.

3.3.1 Vertex-centric

The first intuition for a parallel CPU implementation was to realize a vertex-centric algorithm; this means that each single node represents a task to be elaborated by the worker threads. Nodes are enqueued as tasks by the Main Thread and safely picked up by the workers using a mutex.

Two distinct functions execute the mathematical operations of the message passing flow:

- *workerNode* executes Self-loop, Aggregation, Update and ReLU.

- *workerFinalLayer* is in charge of the Logits' computation.

Both functions share a counter *std::atomic<int> nodes_computed(0)*. This counter is atomically incremented every time a node is successfully processed, avoiding race conditions and corrupted results. Each worker thread reads a copy of the incremented counter into a local variable. If its value is equal to the number of nodes, it means the entire layer is finished. The thread then locks the mutex (with a lock_guard) and notifies the sleeping Main Thread. After waking up, the inference engine checks the current layer type: if it's not the final one, it updates *features vector* with the new values in *next_features* (Double Buffering). Otherwise, the inference engine ends, and the main module calculates the Accuracy.

```
160
161         for (int j=0; j<meta["num_nodes"]; j++)
162         {
163             pool.enqueue([&,j](){workerNode(j, weights);});
164         }
165     }
```

Figure 5 A node enqueued as a task, passing the weights matrix of the current layer

```
138
139     int completati = ++nodes_computed;
140     if (completati == meta["num_nodes"]) {
141         lock_guard<mutex> Lock(layer_mtx);
142         layer_cv.notify_one();
143     }
```

Figure 6 Atomic increment and notify

3.3.2 Vertex-centric with Chunks

During early testing, the vertex-centric implementation with a thread per single node suffered from a performance degradation with respect to the basic sequential one. The cause was the overhead introduced by the thousands of mutex locks that was overwhelming the Thread Pool, and the absurd number of atomic increments on the shared counter, practically killing the parallelism with *bottlenecks*. In other words, the *granularity* was too fine.

In response, a *Chunk-based* solution was implemented. The idea was to use chunks with *variable dimensions*. To have an equally distributed computational cost over all chunks, they needed to have the same amount of operations to process; therefore, the sum of every node plus its neighbors' edges must be roughly equal for every chunk.

To evaluate this workload, only the *indptr* array was used to extract the exact number of neighbours of each node, incremented by one (the node itself):

$$indptr[i+1] - indptr[i] + 1$$

A *threshold* was needed. A first thought was to extract the maximum number of neighbors as the maximum degree, to avoid cutting edges; this would have been ineffective working also with Scale-Free graphs, which tend to be very asymmetrical (*skewed degree distribution*). The risk was to create a few super-chunks, leaving most CPU cores unused and the others overwhelmed with work.

Since the nature of the worker loops is to process all neighbors for each assigned node, no edges are ever broken across different chunks. The adopted solution was instead to decide a target total number of chunks, for example 50. This *dynamic threshold* handles the asymmetrical distribution: if a massive *hub node* (with millions of edges) is encountered, its weight immediately exceeds the threshold. The hub node is assigned to its own dedicated chunk and processed entirely by a single thread, while many isolated nodes are compacted together into other single chunks:

```
163    int n_chunks = 50;
164    int threshold = (meta["num_nodes"] + indptr.back())/n_chunks;
165    int nodes_per_chunk = meta["num_nodes"]/n_chunks;
166    int remainder_nodes = meta["num_nodes"] % n_chunks;
```

Figure 7 Chunks initialization

```
183        for (int j=0; j<meta["num_nodes"]; j++)
184        {
185            current_nodes += 1 + indptr[j+1]-indptr[j];
186            if (current_nodes >= threshold || j == meta["num_nodes"]-1)
187            {
188                end_node = j+1;
189                pool.enqueue([&,start_node,end_node](){workerNodeChunk(start_node,end_node, weights);});
190                start_node = end_node;
191                current_nodes=0;
192            }
193        }
```

Figure 8 Enqueueing of Chunks for Hidden Layers

In the previous snippet of code, the control works in the same way: if the number of nodes in the chunk exceeds the threshold (or if it's the last node of the graph) the end node is set, and the chunk is enqueued. The start node of the following chunk will be the end node of the current one.

For the hidden layers, there's no need to redistribute remainder nodes, since they are also packed in a chunk and enqueued.

For the final layer, instead, every node implies the same computational cost, so the nodes need to be manually distributed across chunks before enqueueing:

```
194        }
195        else
196        {
197            for (int j=0; j<n_chunks; j++)
198            {
199                end_node = start_node + nodes_per_chunk + (j < remainder_nodes ? 1 : 0);
200                pool.enqueue([&,start_node,end_node](){workerFinalLayerNodeChunk(start_node, end_node);});
201                start_node = end_node;
202            }
```

Figure 9 Enqueueing of Chunks for Final Layer

Finally, instead of performing an atomic add for each node elaborated (which destroys the performance), with this solution the counter is incremented just once per chunk, which results in just 50 atomic increments:

$$nodes_computed \mathrel{+}= (end - start)$$

3.3.3 Edge-centric with Chunks

For this parallelization strategy, the working unit is no longer the node but the *edge*. This solution completely decouples the workload distribution from the graph's topology, avoiding bottlenecks caused by hub nodes. For this approach, the concepts of 'source node' and 'destination node' need to be revised. Considering a specific node in the topology:

- A *Source node* is every node listed as a neighbor of the considered node in the *indices* array.
- The *Destination node* is the current node, since it is receiving the messages for aggregation from all its neighbors.

To exploit this organization, the *edge_dest* array was created with the same size as the *indices* array. It is filled within an outer loop over all the nodes and an inner loop on their incoming edges, using the *indptr* array. Calling *i* the current node, and *j* the edge index, *edge_dest[j]=i*.

```
90  vector<int> edge_dest_compute(const vector<int>& indices, const vector<int>& indptr)
91  {
92      vector<int> edge_dest(indices.size());
93      for (int i=0; i<meta["num_nodes"]; i++)
94      {
95          for (int j=indptr[i]; j<indptr[i+1]; j++)
96          {
97              edge_dest[j] = i;
98          }
99      }
100     return edge_dest;
101 }
```

Figure 10 Edge_dest array computation

The message passing computation for Hidden Layers is split into two different but synchronized phases with two global barriers (using atomic counters *edges_computed* and *nodes_computed*):

- **Edge-Parallel Aggregation**. Since every edge represents exactly one mathematical operation (the sum of a neighbor's feature vector), the workload can be perfectly balanced by statically dividing edges into equal-sized chunks.

  A support *aggregated_features* array was needed because, unlike the vertex-centric strategy (which aggregates all neighbors of a single node in a local variable and writes the result in *next_features* in one definitive step), here edges are independently processed by different threads during this phase. This global array is necessary to safely save the partial sums computed for every edge processed. Only when phase 1 has finished, the next phase can use *aggregated_features* to compute the matrix multiplication. On the other hand, this array was highly exposed to race conditions, being accessed by multiple working threads simultaneously. To prevent data corruption without causing massive bottlenecks using a mutex inside the loop, the C++20 *std::atomic_ref<float>* wrapper was used. This tool allows the engine to treat every float inside the array as an atomic variable, performing an atomic addition on the specific memory cell, maintaining high performance and extremely low overhead.

```
108     for(int i=start; i<end; i++)
109     {
110         int dst=edge_dest[i];
111         int src=indices[i];
112
113         for (int j=0; j<num_features; j++)
114         {
115             std::atomic_ref<float> cell(aggregated_features[dst*num_features+j]);
116             cell += features[src*num_features+j];
117         }
118     }
```

Figure 11 Use of the C++20 Wrapper

- **Vertex-parallel Update and ReLU**. Once all edges have been processed by the first phase (and the global barrier *edges_computed* is passed), it means the aggregation is completed. The Update phase doesn't depend on the edges, but only on the nodes' aggregated states; therefore node-chunks are enqueued (as in the Vertex-centric strategy with Chunks). Here, the *next_features* array is computed exploiting the raw aggregated values saved in the *aggregated_features* array.

```
209         aggregated_features = features;
210
211         for (int j=0; j<n_chunks; j++)
212         {
213             end_edge = start_edge + edges_per_chunk + (j < remainder_edges ? 1 : 0);
214             pool.enqueue([&,start_edge,end_edge](){workerAggEdgeNode(start_edge, end_edge);});
215             start_edge = end_edge;
216         }
217
218         {
219             unique_lock<mutex> Lock(layer_mtx);
220             layer_cv.wait(Lock, [&]() { return edges_computed == meta["num_edges"];});
221             edges_computed=0;
222         }
223
224         for (int j=0; j<n_chunks; j++)
225         {
226             end_node = start_node + node_per_chunk + (j < remainder_nodes ? 1 : 0);
227             pool.enqueue([&,start_node,end_node](){workerUpdateEdgeNode(start_node, end_node, weights);});
228             start_node = end_node;
229         }
```

Figure 12 Two phases of the Message Passing flow

The **Final Layer** also uses a vertex-centric with chunks approach, since no edge aggregation is needed.

The self-loop is simply implemented by initializing the *aggregated_features* array with the *features* array.

An asynchronous approach was also considered: updating each node state as soon as all its incoming edges are processed, without having to wait for the global barrier. This strategy was discarded because it required an atomic counter for every single node (to check if all incoming edges have been processed), introducing an unfeasible overhead, making the global barrier the cleaner solution.

3.4 Parallel GPU Implementation

To implement the parallel GPU versions of the inference engine, the NVIDIA CUDA architecture was used. Unlike the CPU environment, which required the manual initialization of a Thread Pool, the GPU relies on a massive parallelism obtained by distributing the workload across thousands of threads, grouped into Warps and Blocks.

The main goal to get higher performance while elaborating a GNN topology on the GPU is to achieve *Coalesced Memory Accesses* (which means ensuring that adjacent threads in a Warp access contiguous memory addresses, to group them into a single and fast memory transaction), also maximizing the L1 Cache Hit Rate and minimizing the Global Memory traffic.

The initialization phase transfers the graph's topology, features and weights to the Device VRAM using *cudaMemcpy*. As in the CPU implementation, the Hidden Layers exploit a *Double Buffering* technique to store raw and intermediate results, to then perform a pointer swap, having the features vector pointing to the new states of the nodes.

```
116             float* temp = d_features;
117             d_features = d_next_features;
118             d_next_features = temp;
```

Figure 13 Pointers Swap

At the end of the inference engine, a single *cudaMemcpy DeviceToHost* is performed, to copy the computed logits onto the host before freeing the VRAM with *cudaFree*.

Each parallelization strategy requires different mapping of the workload; therefore, the dimensions of the CUDA grid and Blocks were customized for each implementation, assigning threads to nodes, edges, or features. In particular, the strategies implemented were:

- **Thread per Node with Global Memory**
- **Thread per Feature with Shared Memory**
- **Thread per Feature with Shared Memory and Feature Compression**
- **Thread per Edge with Global Memory**

3.4.1 Thread per Node with Global Memory

The first intuition for a parallel GPU implementation was to translate the CPU vertex-centric algorithm to CUDA architecture. Here, a thread is assigned to exactly one node of the graph.

The CUDA grid has been instantiated as a 1D layout of blocks, and the dimension of blocks was set at the standard number 256. To evaluate the total number of blocks (which corresponds to the grid dimension), the following formula was used:

$$Number\ of\ blocks = \frac{Number\ of\ nodes + Dimension\ of\ block - 1}{Dimension\ of\ block}$$

This calculation allows to obtain an integer number of blocks, rounded up to ensure the elaboration of every node.

```
96      int num_blocks = (num_nodes+DIM_BLOCK-1)/DIM_BLOCK;
97      dim3 dim_grid(num_blocks, 1, 1);
98      dim3 dim_block(DIM_BLOCK, 1, 1);
```

Figure 14 CUDA layout instantiation

Two *kernels* were needed for this implementation:

- **node_compute** is the function that executes the calculation of the Hidden layers. It implements the self-loop of each node (thread), the Aggregation of the neighbors' message (using *indptr* array), and applies the ReLU activation function.

  Speaking of race conditions, there is no need to use atomic operations on the *d_next_features* array, since every node operates independently.

  This first approach suffers from workload imbalance, since working with highly skewed degrees Scale-free graphs, a thread assigned to a hub node would take an unbalanced amount of time to aggregate all its neighbours, leaving the other threads to sit idle in the Warp.

  Furthermore, this implementation only exploits the slow Global Memory. The usage of the Shared Memory would have been useless: since each thread independently processes a different node, there is no collaboration among threads within the same Block, and no justification to implement a shared cache.

```
11      int node_idx=blockIdx.x*blockDim.x+threadIdx.x;
12      if (node_idx >= num_nodes) return;
13
14      for (int i=0; i<num_features; i++)
15      {
16          d_vsum[node_idx*num_features+i] = d_features[node_idx*num_features+i];
17      }
18
19      for (int i=d_indptr[node_idx]; i<d_indptr[node_idx+1]; i++)
20      {
21          for (int k=0; k<num_features; k++)
22          {
23              d_vsum[node_idx*num_features+k]+= d_features[d_indices[i]*num_features+k];
24          }
25      }
```

Figure 15 Thread ID computation and Aggregation

- **node_compute_final** is the kernel in charge of the last layer of the network. Since no aggregation is required for this layer, the function simply performs the multiplication between the *weights_out matrix* and the *d_features array*, generating the final *logits*.

3.4.2 Thread per Feature with Shared Memory

To overcome the problem of the unbalanced workload introduced by the previous approach, the *Thread-per-Feature* solution was explored. In this case, instead of assigning a whole node to a single thread, an entire CUDA Block is assigned to a node, and each thread within the block is responsible for computing a single feature of that node. Practically, the dimension of the grid is equal to the number of nodes, and every block contains as many threads as the number of features of a single node.

This implementation also needed the definition of a new grid dimension to launch the Final Layer kernel. Since one thread represents one output cell, for the logits' computation only *num_classes* cells are needed for each node:

```
97      int num_blocks = num_nodes;
98      dim3 dim_grid(num_blocks, 1, 1);
99      dim3 dim_block(num_features, 1, 1);
100     dim3 dim_block_final(num_classes, 1, 1);
```

Figure 16 CUDA layout instantiation

Two *kernels* were designed for this implementation:

- **feature_compute** is the function that executes the calculations for the Hidden Layers of the network. This kernel introduces the main optimization of this approach: the Shared Memory. It is an on-chip programmable cache that provides extremely low-latency memory access.

  Because the number of features is not a fixed value, but depends on the input dataset, the allocation of the Shared Memory is dynamic:

  *size_t shared_bytes = num_features * sizeof(float);*

  The *shared_bytes* value also needs to be passed as third parameter to the kernel:

  feature_compute<<<dim_grid, dim_block, shared_bytes>>>(d_features, …)

Inside the Kernel, the shared memory is instantiated by *extern __shared__ float s_vsum[]*. The *s_vsum array* is the private memory of a block, hence a specific node. Since the size of *s_vsum* is equal to the number of features, each thread (which represents a feature of a specific node) can safely access its own cell *s_vsum[threadIdx.x]*. Unlike the Global Memory, the shared one is limited to the Block's boundaries, so every Block gets a newly instantiated and isolated *s_vsum* array. This approach makes the memory access much simpler and faster.

During the Aggregation phase, all threads in the block simultaneously read the features of a neighbor node from the Global Memory and accumulate them in the *shared array*, using *Coalesced Memory Accesses.*

Before proceeding with the Update phase, all threads in a block must have finished the Aggregation, because the shared array needs to contain the full aggregated message; therefore, immediately after the Aggregation loop, a *__syncthreads()* barrier has been used.

```
10    int feature_idx=blockIdx.x*blockDim.x+threadIdx.x;
11
12    if (feature_idx >= num_features*num_nodes) return;
13
14    extern __shared__ float s_vsum[];
15
16    int node_idx = blockIdx.x;
17
18    s_vsum[threadIdx.x] = d_features[feature_idx];
19
20    for (int i=d_indptr[node_idx]; i<d_indptr[node_idx+1]; i++)
21    {
22
23        s_vsum[threadIdx.x]+= d_features[d_indices[i]*num_features+threadIdx.x];
24
25    }
26
27    __syncthreads();
```

Figure 17 Shared Memory, Aggregation and barrier

After the barrier, the Update phase takes place. Unlike the *Thread per Node* strategy (where a single thread must multiply the entire *weights matrix* by the *d_vsum* array), here each thread only needs to multiply the aggregated array by the *weights matrix* column corresponding to its assigned feature. Since this aggregated message is now cached in the Shared array, these heavy memory reads are performed at extremely high speed on-chip, bypassing the slow Global Memory. Finally, the ReLU activation is applied, and the result is stored back into Global Memory.

```
34    float sum=0.0f;
35    for (int k=0; k<num_features; k++)
36        {
37            sum+= d_weights[threadIdx.x*num_features+k]*s_vsum[k];
38        }
39
40        d_next_features[feature_idx]=fmaxf(0.0f, sum);
41
```

Figure 18 Feature Update

- **logit_compute_final** is the kernel that computes the last layer of the network. As said before, it requires a different grid dimension (number of classes), since it's in charge of calculating logits. Here the logic switches from Thread per Feature to *Thread per Logit cell*. Using a thread-per-feature approach even in this phase would have meant waiting for all features of a node to be summed up, thus overwriting the result of that logit cell *num_features* times,

requiring atomic operations to avoid the race condition caused by all threads trying to write on that same logit cell.

3.4.3 Thread per Feature with Shared Memory and Feature Compression

The final optimization aims to reduce the *Memory Footprint* of the dataset's features. Using 32-bit float arrays limits the bandwidth utilization; therefore, to double the reading speed from the Global Memory, the *<cuda_fp16.h> library* was used, enabling the use of *half-precision* float numbers.

In the Host *inference_engine* function, the *float** pointers for the *d_features* and *d_next_features* vectors have been replaced with *__half** pointers.

```
72      __half* d_features;
73      __half* d_next_features;
74      float* d_weights_out;
75      float* d_logits;
76      int* d_indices;
77      int* d_indptr;
```

Figure 19 New pointers

For this reason, the *cudaMalloc* allocations on the VRAM for those vectors occupy half the bytes (compared to the previous implementation).

```
81      cudaMalloc((void**)&d_features, num_nodes * num_features * sizeof(__half));
82      cudaMalloc((void**)&d_next_features, num_nodes * num_features * sizeof(__half));
```

Figure 20 VRAM allocations

For this implementation, three *kernels* were needed:

- **cast_to_half** is a support kernel to convert float numbers into half-precision float numbers. It takes the *d_features_float* array as input and returns the *d_features* array which contains the input array's cells cast to half-precision with the *__float2half* method.

```
11  __global__ void cast_to_half(const float* in, __half* out, int size) {
12      int idx = blockIdx.x * blockDim.x + threadIdx.x;
13      if(idx < size) {
14          out[idx] = __float2half(in[idx]);
15      }
16  }
```

Figure 21 Cast to half kernel

- **feature_compute_compressed** is the function that computes the Hidden Layers. It remains identical to the previous implementation (*Thread per Feature with Shared Memory*), but with special care for computational precision. In fact, the Shared Memory is still instantiated with a 32-bit float data type, and the features values for the Self-loop and the Aggregation are cast back to float with the *__half2float* method before being accumulated into the *s_vsum* array. This passage preserves the mathematical precision of the computation.

```
23      extern __shared__ float s_vsum[];
24      int node_idx = blockIdx.x;
25
26      s_vsum[threadIdx.x] = __half2float(d_features[feature_idx]);
27
28      for (int i = d_indptr[node_idx]; i < d_indptr[node_idx+1]; i++)
29      {
30          s_vsum[threadIdx.x] += __half2float(d_features[d_indices[i] * num_features + threadIdx.x]);
31      }
```

Figure 22 Cast to 32-bit float

The usage of mixed data types avoids bottlenecks in the Global Memory, still allowing the GPU to perform full 32-bit precision operations. Finally, before writing the Update result back into the Global Memory, it is compressed again into 16-bit float values, together with the ReLU:

d_next_features[feature_idx] = __float2half(fmaxf(0.0f, sum));

- **logit_compute_final_compressed** is the Final Layer kernel. Its implementation is the exact same as the previous parallelization strategy, with the additional step of a half-cast on the features' values:

  *sum += d_weights_out[...] * __half2float(...);*

  There's no need to compress the computed Logits, since the majority of the bottleneck problem affects the Hidden Layers.

3.4.4 Thread per Edge with Global Memory

Theoretically, an *Edge-centric* parallelization approach, assigning a thread to a single graph's edge, is an optimal way to perfectly balance the Workload across the GPU, having every thread perform exactly one operation.

However, analysing this architectural choice, it was clearly noticeable that a *Thread per Edge* strategy would not have led to a performance increase. Despite this observation, this parallel GPU version was implemented for experimental purposes, to show its limitations also in the numerical evaluation.

The Block size is set at the default value 256, and the CUDA layout instantiation is performed as follows:

```
109     int num_blocks = (meta["num_edges"]+BLOCK_SIZE-1)/BLOCK_SIZE;
110     dim3 dim_grid(num_blocks, 1, 1);
111     dim3 dim_block(BLOCK_SIZE, 1, 1);
112     dim3 dim_block_final(num_classes, 1, 1);
113     dim3 dim_grid_update(num_nodes, 1, 1);
114     dim3 dim_block_update(num_features, 1, 1);
```

Figure 23 CUDA layout instantiation (edges)

For this implementation, the Hidden Layer computation had to be split into two functions. Therefore, three kernels were needed:

- **edge_agg** is the Aggregation kernel. An additional array *d_edge_dest* was passed to the GPU to map each edge to its destination node. The thread reads the features of the source node and accumulates them into the destination node's temporary array *d_vsum*.

```
12      int edge_idx=blockIdx.x*blockDim.x+threadIdx.x;
13
14      if (edge_idx >= num_edges) return;
15
16      int dest = d_edge_dest[edge_idx];
17      int source = d_indices[edge_idx];
18
19      for (int i=0; i<num_features; i++)
20      {
21          atomicAdd(&d_vsum[dest*num_features+i], d_features[source*num_features+i]);
22      }
23
```

Figure 24 Edge Aggregation

- **feature_update**. Once the Aggregation over edges is complete, this function reverts to a *Thread per Feature* approach. The reason behind this switch is purely mathematical: at this stage, edges are no longer needed, because the Update and the ReLU are operations executed on the newly aggregated message.
- **logit_compute_final** is the kernel to compute the Final Layer. It remains identical to the previous implementation.

As predicted, this model already showed critical issues during early testing. In particular:

- *Impossibility to exploit a Shared Memory*. Unlike the *Thread per Feature* architecture, here threads within the same Block are not assured to be neighbors, so they don't have any data to share within the block. Because there is no data locality (and consequently no possibility for a *Coalesced Memory Access*), the only solution is to use the Global Memory. This sparse pattern drastically degrades the Memory performance.
- *Race conditions and AtomicAdd bottleneck*. Every time a destination node has more than one neighbor, multiple threads will try to simultaneously write their aggregated features into its memory area, causing a Race Condition. To prevent data corruption, the *atomicAdd* instruction was used.

  Atomic operations force the GPU to serialize the memory accesses: threads have to wait for their turn to write into a memory cell. The worst scenario occurs when using Scale-free networks, where there are potentially many hub nodes (popular, with lots of neighbors). The use of atomic operations destroys the GPU parallelism, putting threads into a queue.

4. Experimental Evaluation

4.1 Experimental Setup

4.1.1 Hardware Specifications

All the implementations were run on a laptop with the following hardware specifications:

- CPU: AMD Ryzen 7 7745HX with 8 physical cores and 16 logical threads.
- GPU: NVIDIA GeForce RTX 4070 (equipped with 8 GB of VRAM).
- RAM: 16 GB DDR5.

4.1.2 Profiling Tools & Metrics

NVIDIA Nsight Systems and *NVIDIA Nsight Compute* were used to collect all the data presented in the experimental evaluation:

- **Execution Time**: the inference time, measured in seconds (CPU) or milliseconds (GPU). NVTX Markers were used to isolate the inference time, ensuring that dataset loading and metric evaluation phases were excluded from the measurement.
- **Mem Busy (%):** represents the throughput of internal activity (amount of transferred data requested by the GPU) within caches and DRAM as a percentage of the peak sustained rate.
- **L1 Hit Rate (%)**: the percentage of memory requests successfully served by the fast on-chip L1 cache. This metric demonstrates how the memory traffic is handled (high values mean low traffic on VRAM).
- **Max Bandwidth (%)**: represents the percentage of the peak sustained rate achieved for memory requests.
- **Memory Throughput (GB/s)**: the total number of bytes accessed in the device memory (DRAM) per second.
- **Memory Footprint (GB)**: the total amount of device memory allocated to store the graph topology and the node features.

***Official definitions from NVIDIA Nsight Compute*

4.2 Global Performance Analysis

4.2.1 CPU vs GPU: Global Speedup

Table 1 Global Evaluation Throughput (Nodes/ms)

	CPU IMPLEMENTATIONS				GPU IMPLEMENTATIONS			
DATASET \ STRATEGY	*Sequential*	*Node*	*Node (Chunks)*	*Edge*	*Edge*	*Node*	*Features*	*Features (FP16)*
Scale free (2.5 mln)	41,55	27,60	362,68	215,31	917,48	836,12	1.309,57	1.328,47
Small-world (1.6 mln)	57,42	13,70	340,86	237,38	1022,46	868,89	1.336,75	1.323,10
Random (300000)	55,23	23,65	354,10	252,10	942,62	816,88	1.333,57	1.329,37
Ogbn-arxiv (169343)	59,92	13,76	369,08	305,55	1.064,04	878,24	1.226,94	1.285,24

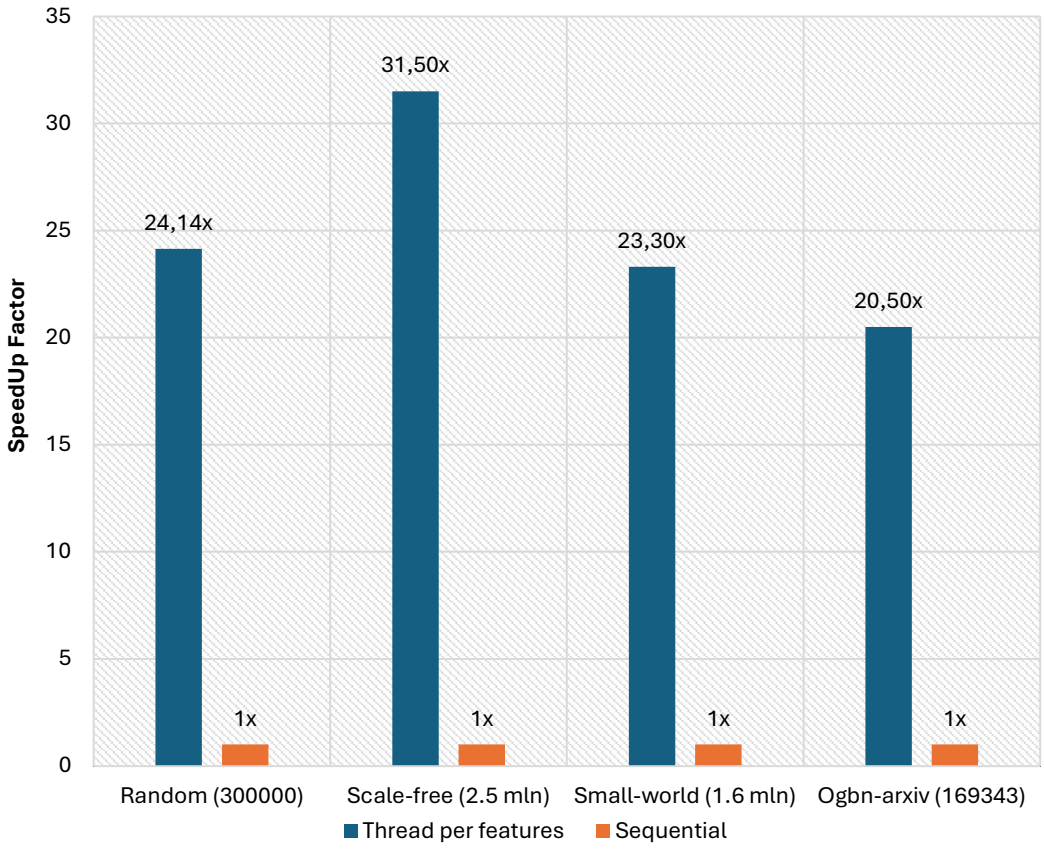


Figure 25 Global Speedup of the Feature-centric GPU architecture over the Sequential CPU

Table 1 shows the Throughput (expressed in processed nodes per millisecond) across all implemented solutions. The data demonstrates the high robustness of the implementations, since the inference engine throughput for each parallelization strategy remains substantially constant for all different graphs' topologies and number of nodes tested.

The *Speed Up* chart (*Figure 25*) compares the absolute best-performing non-compressed approach (*GPU Thread per Features*) against the baseline *Sequential CPU* implementation, over the four types of graphs. This data demonstrates the massive acceleration introduced by the GPU parallelization, ranging from 20,50x Speed Up on the *ogbn-arxiv* real dataset, to a peak of 31,50x on the *Scale-Free* synthetic graph.

The Speed Up gap between these two architectures is explained by their hardware models. In the Sequential version, the entire computation (both Aggregation and Update) is executed on a single CPU core using heavy and slow synchronous nested loops. In the Thread per Feature solution instead, all features of a single node complete the Aggregation phase simultaneously, since they are mapped one-to-one to the threads of the same CUDA block. Once the Aggregation is over, each thread accesses the Shared Memory and updates its assigned feature in parallel.

4.2.2 CPU Parallelization Strategies

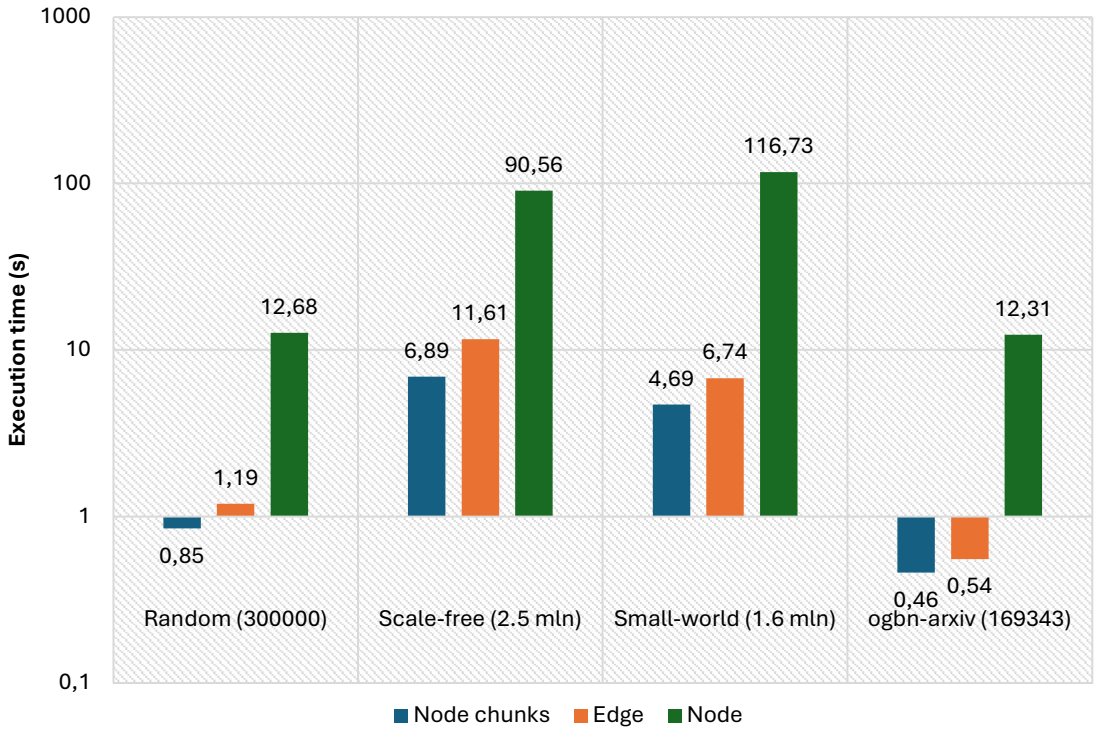


Figure 26 CPU Multi-core execution time comparison (Node vs Node-chunks vs Edge)

Focusing on the multi-core CPU architectures, *Figure 26* illustrates the trend of execution times on a logarithmic scale. The pure *vertex-centric* mathematical approach ('*Node*') proves to be by far the slowest across all tested datasets. Meanwhile, the '*Node Chunks'* and '*Edge*' versions drastically reduce execution times (achieving up to a 10x speed-up compared to the pure Node approach), guaranteeing significantly lower latencies. Furthermore, the chart demonstrates how the usage of Chunks to balance the workload among threads is successful, registering the lowest execution times

for all datasets. The Dynamic Chunking mechanism prevents bottlenecks in the task queue, which would otherwise be overloaded by individual nodes; in this system, worker threads can operate continuously for longer time spans, avoiding the overhead of sleep and wake-up cycles.

The CPU *edge-centric* implementation performs particularly well despite relying heavily on *std::atomic_ref* operations. In a GPU environment, this massive usage of atomic operations would destroy parallelism. However, in the CPU scenario, the benefits of a perfectly balanced workload outweigh the minimal overhead introduced by atomics, which are well handled by the powerful cores and extremely fast caches.

4.3 GPU Architecture Evaluation

4.3.1 Vertex-centric vs. Edge-centric

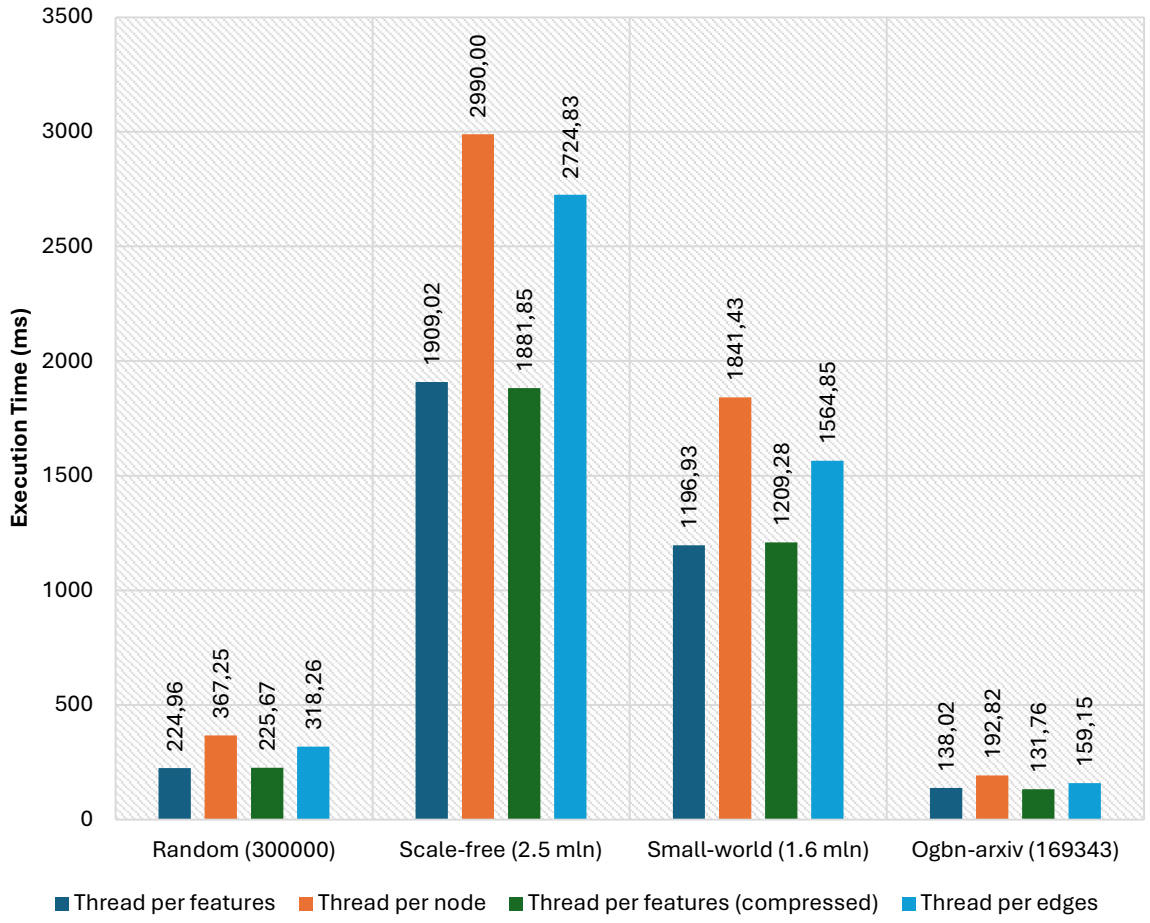


Figure 27 GPU execution time comparison across different parallelization strategies

Figure 27 measures the total execution time across different CUDA thread assignment granularities. Both *feature-centric* versions (standard FP32 and FP16 compressed) yield the lowest inference times, standing out as the fastest approaches in all tested scenarios.

Assigning threads to single *nodes* or *edges* results in remarkably degraded performances and extended execution times. In particular, the performance of the *edge-centric* strategy drops drastically due to the massive use of the *atomicAdd* operation; in fact, hub nodes trigger a sequence of atomic operations

on every feature that serializes the process and creates severe bottlenecks. The chart also highlights that assigning a single thread per node is highly inefficient. As opposed to the *feature-centric* strategy (which distributes the mathematical operations across multiple threads), the *node-centric* one forces a single thread to execute the nested loops over all features and edges. This makes a large portion of the computation strictly sequential for each node. Furthermore, this leads to severe workload imbalance on Scale-Free graphs: threads assigned to massive hub nodes remain stuck executing these long sequential loops, while the others sit idle waiting for them. Overall, this strategy does not fully exploit the parallel nature of the GPU. In the *feature-centric* approach, these bottlenecks are completely avoided, and the usage of Shared Memory further minimizes inference times.

4.3.2 Performance and Memory Footprint Evaluation

Table 2 GPU Architecture Profiling on Scale-free Dataset (2.5M nodes, 128 features)

GPU STRATEGY \ METRIC	*Execution Time (ms)*	*L1 Hit Rate %*	*Max Bandwidth (%)*	*Mem Busy %*	*Memory Throughput (GB/s)*	*Memory Footprint (GB)*
Thread per features (shared memory)	1909,02	98,42	9,42	98,19	24,10	1,28
Thread per features compressed (shared memory)	1881,85	99,14	8,87	98,21	15,76	0,64
Thread per nodes (global memory)	2990,00	72,27	58,74	84,99	30,41	1,28
Thread per edges (global memory)	2724,83	28,35	56,14	56,31	43,13	1,28

Table 2 shows the differences in memory across the tested approaches. The *feature-centric* strategy reaches an *L1 Hit rate* of 98-99%, against the drastically lower values obtained by the *edge-centric* implementation (28%).

The usage of the *FP16 Feature Compression* technique halves the *VRAM Footprint* required to store the *feature array*, going from 1,28 GB to 0,64 GB.

The *feature-centric* implementation uses blocks having dimension equal to the number of features per node; this approach combined with the usage of the Shared Memory generates *Coalesced Memory Access* patterns, allowing optimized memory transactions.

The extremely high Hit Rate for both *feature-centric* implementations (with and without Compression) demonstrates how most data used by GPU *kernels* was already stored in the Shared Memory cache, without having to access the slow Global Memory. The remaining ~2% of memory requests (cache misses) are satisfied by the VRAM; consequently, the *Bandwidth* utilization of the Global Memory is low, around 9%. Since the *Mem Busy* metric includes both the VRAM and the L1 cache traffic, the high Hit Rate percentage is reflected also by the Mem Busy one.

4.4 Scalability Evaluation

4.4.1 Number of Features

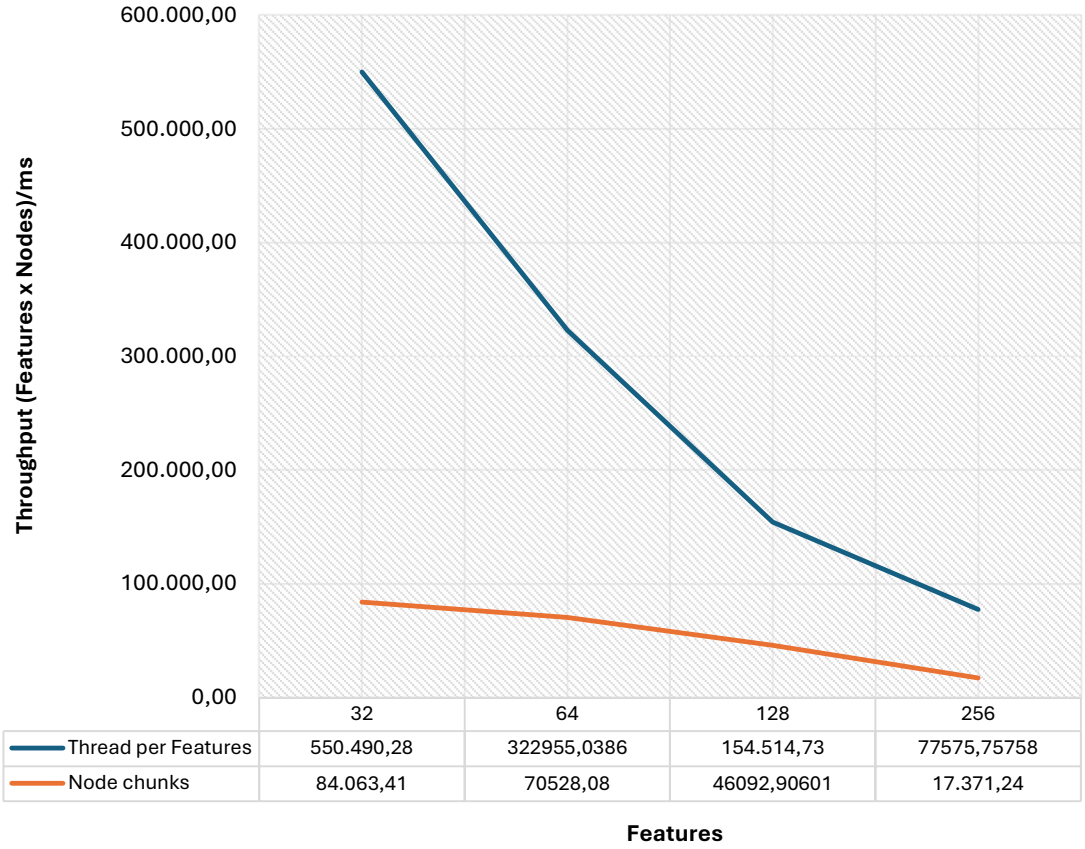


Figure 28 Throughput scalability evaluating the number of features on a Small-world graph (1M nodes)

Figure 28 shows how the number of features affects the Throughput. Scaling the number of features by a factor of 8 (from 32 to 256) both GPU and CPU Throughput, as expected, drastically decreases. By increasing the number of features, the operations performed by each thread grows in number; denoting *k* the scaling factor of the *feature array* dimension, the number of multiplications required by the *weight matrix* grows by *k^2.* For example, going from 32 to 256 features, the scale factor is equal to 8, and the effective computational workload per node grows by 64 times.

4.4.2 Number of Nodes

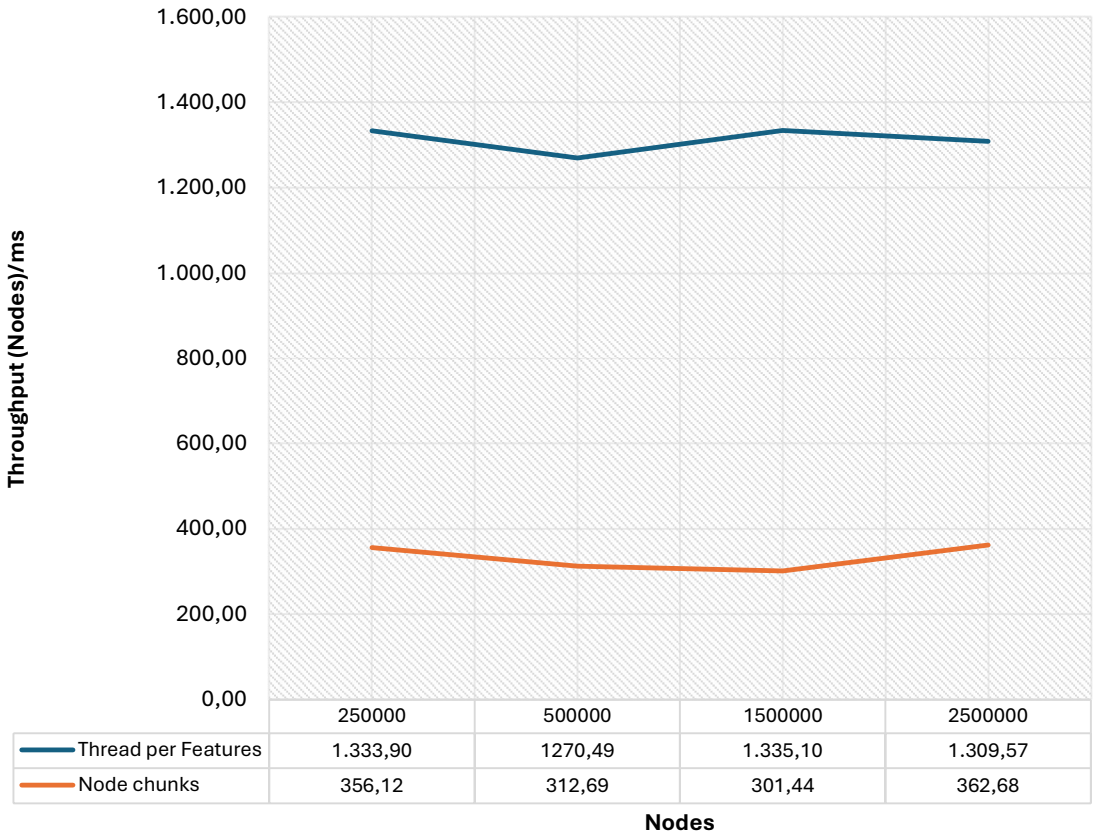


Figure 29 Throughput scalability evaluating the number of nodes on a Scale-free graph (128 features)

Figure 29 demonstrates the correlation between the number of nodes and the overall Throughput. Increasing the number of the graph's nodes from 250.000 to 2.5 mln, the Throughput stabilizes into an almost flat line. This happens because the effective computation times per thread remain almost unchanged, while the number of CPU tasks or GPU threads grows.

The ratio between the number of nodes and the total inference time stays stable because both operands increase by the same factor.

5. Conclusions

This project developed and evaluated a working GNN inference engine, exploring the architectural differences between CPU and GPU parallelization strategies. The experimental evaluation clearly showed how the algorithmic logic and the underlying hardware execution model need to be aligned in order to obtain high performance.

On the CPU, dynamic chunking techniques successfully balanced the workload and mitigated the overhead of context switching. This solution permits the *edge-centric* design to be highly competitive despite its heavy reliance on atomic operations. The GPU potential was fully exploited by the *Thread per Feature with Shared Memory* implementation. This approach overcame the skewed degree

distribution introduced by the Scale-free graph and avoided the serialization bottlenecks of *atomicAdd* operations.

Finally, this project demonstrates that not all scenarios can be solved with a single approach: the optimal parallelization strategy for a GNN inference engine is strictly dependent on both the hardware architecture and the graph topology.